Problem Statement

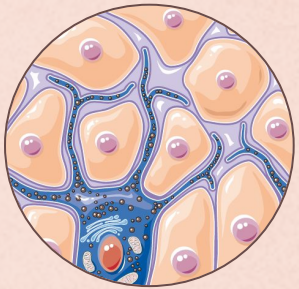
What is colon cancer?

Colon cancer (or colorectal cancer) is cancer that develops in the tissues of the colon or rectum.

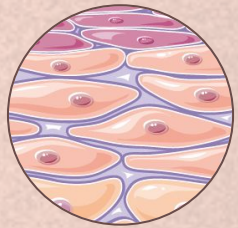
The Impact

There are rising rates of young people getting colon cancer. According to American Cancer Society, 20% of diagnoses in 2019 were in patients under 55, which is double the rate from 1995.

The Problem



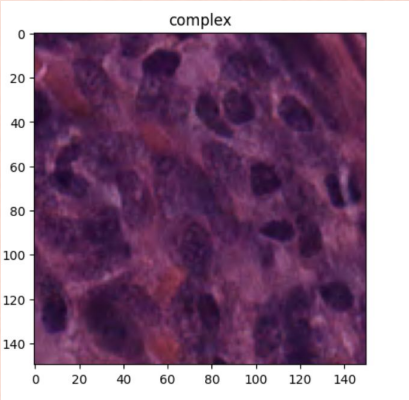
Studies aiming to better understand colon cancer require many labeled microscopic images of colorectal tissues. This is labor-intensive and varies for pathologists, so having AI to be able to automatically classify colorectal tissue types could make the process easier and reduce diagnostic variability.

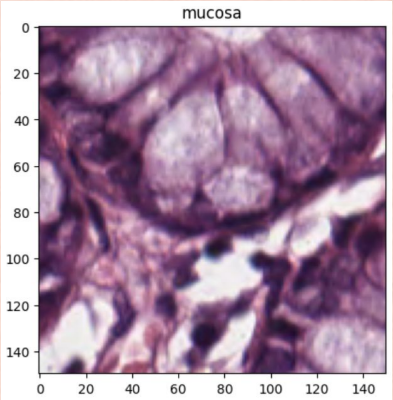


Dataset & Data Processing

Dataset

Our dataset uses a total of 1024 images, each with the dimension of 150 pixels x 150 pixels. These images depicts the tissues types which includes: mucosa - 144, tumor - 144, adipose - 141, stroma - 126, lympho - 122, debris - 120, empty- 115, complex - 112





Data Processing

With these images that are ready to be used, one technique that we have chosen to use to process our image is one hot encoding. A process where each of these images are being categorize and can be represented in a vector or a list.

```
                0  1  2  ...
category 0 -> [1, 0, 0, ...]
category 1 -> [0, 1, 0, ...]
category 2 -> [0, 0, 1, ...]
...
```

CNN Background

Neural Networks

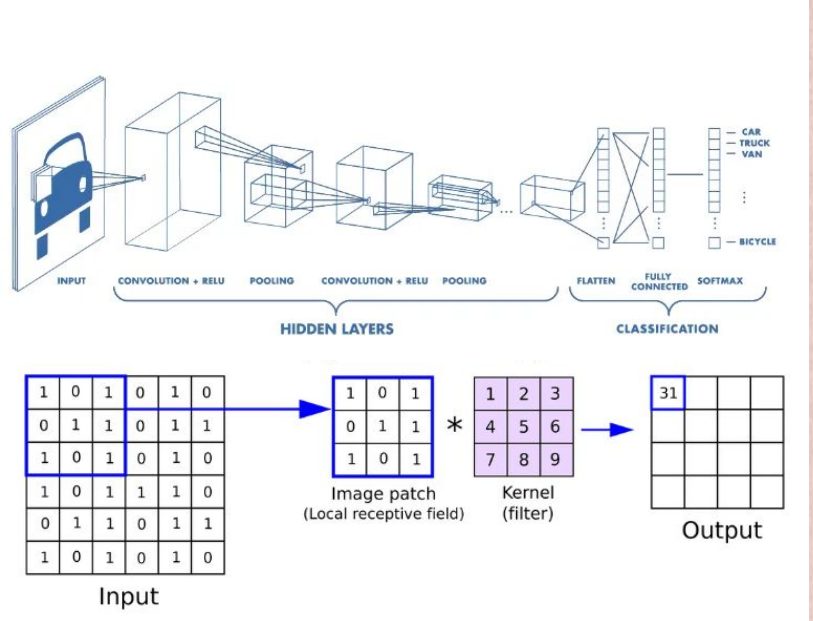
- An algorithm loosely inspired by the structure of the brain
- Computer vision
  - Images → numerical values
  - Placing weights on inputs

CNNS

- Able to recognize complex patterns within inputs
  - Kernels through multiple layers
- Detect a pattern in a input in multiple locations at once

Pros and Cons

✔ learn important features of an input automatically
✖ require large labeled datasets for accuracy, prone to overfitting



Our CNN Model

Input Layer

The input layer will take in a 150x150 pixel image with the 3 color channels of Red, Green, and Blue.

Convolutional Layer

5 layers of convolutions to the model, with 3x3 kernels, alternating between 32 and 16 convolutions per layer. Features a relu activation function and a max pooling using a 2x2 pool size.

Flattening Layer

Flattening the outputs of the last convolution into a single array of features. This transforms a 3D matrix into a 1-dimensional array.

Fully Connected Layer

The single array of features is passed onto the fully connected/dense hidden Neural Network which has 64 nodes and a relu activation function.

Output Layer

A final layer that has 8 nodes representing the 8 different features/labels, featuring a softmax activation function in order to predict the scores for each of the categories in the dataset.

Keras

This model was built using Keras, an open-source library that provides a Python interface for artificial neural network, it integrated into the TensorFlow library, which is an open-source software library for machine learning and artificial intelligence.

CNN Training Process

Compiling

This CNN mode was compiled with a loss function of categorical cross entropy, an optimizer of rmsprop, and tracks the metrics of accuracy and categorical_crossentropy.

Training

The dataset was split into X_train and y_train as well as X_test and y_test. 80% of the dataset was used for training, while the other 20% was used for testing. The model was trained with the data values of X_train and y_train, for 10 epochs, with a batch size of 64, and the data for validation being the test data, or X_test and y_test.

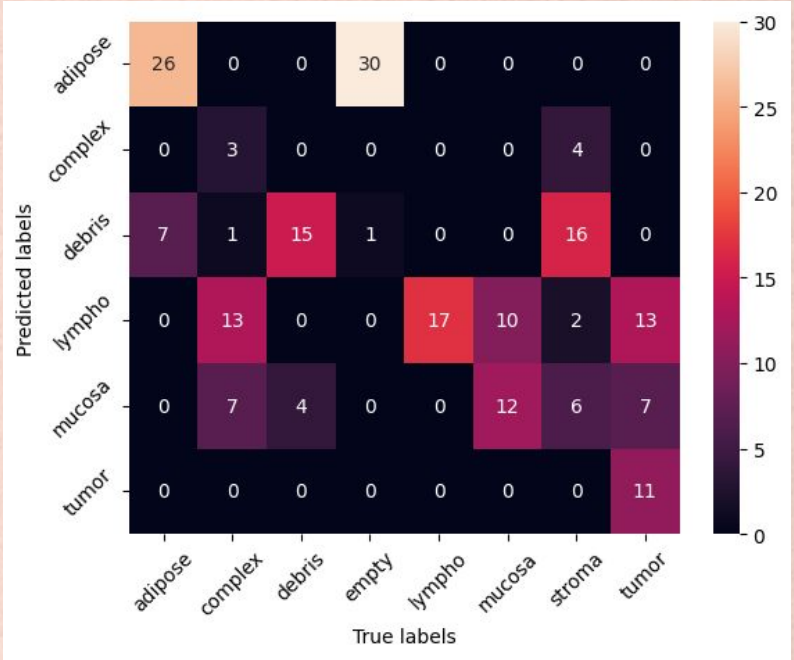
Model Results

Accuracy on test data = 41%

- Misclassifications for all labels
  - Significant in complex, debris, lympho, and mucosa

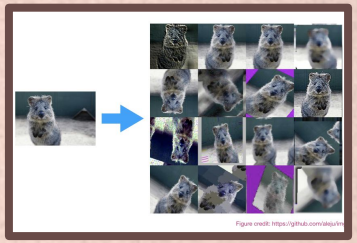
Cause

- Small data set (less diverse examples)
  - overfitting



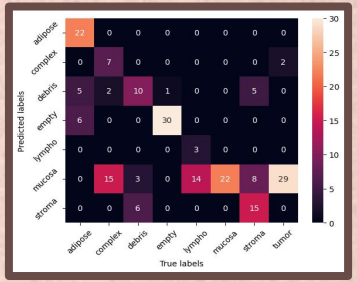
Data Augmentation

An example of data augmentation



Since our model only had around 800 training images and 200 testing images, the accuracy was pretty bad as the CNN lacked data. We used Data augmentation to fix this. **Data Augmentation** is when we transform a single image into multiple via transformations such as cropping, rotating, flipping, changing the contrast, etc.

Confusion matrix for augmented data



These augmented images greatly increased the number of training data that we had, and helped out model accuracy. Previously, our model accuracy on the test data was 41%, but after the data augmentation it was 53.1% accurate! Though this still could use some work, it is a significant upgrade.

Transfer Learning

Challenges with Training from Scratch

- Time and resource consuming
- High accuracy is difficult
- Need large datasets

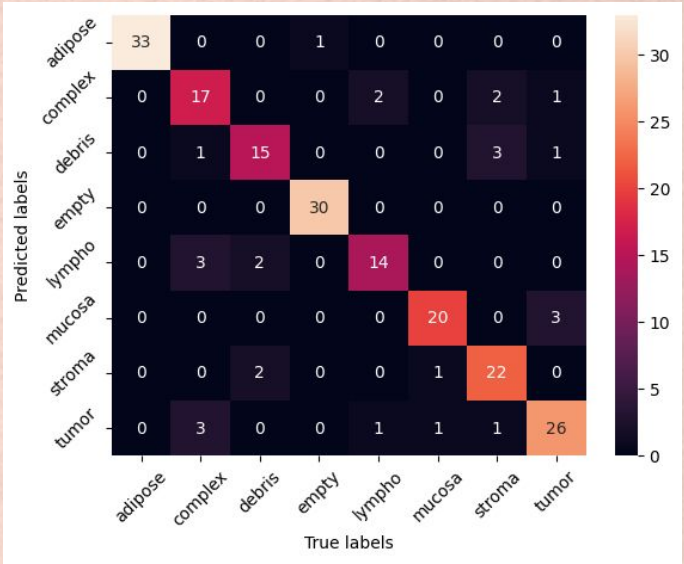
To combat this, we can use transfer learning: reusing pre-trained models and modifying them to our needs. We used the model VGG16 which is trained on millions of images and thousands of classifications

Applying the model

1. Modify the output layer to have 8 nodes
2. Freeze the weights in other layers
3. Resize images to be compatible with the input layer
4. Train the model

Results

Accuracy jumped to 86.3%
Less confusion (confusion matrix below)



Model Comparison

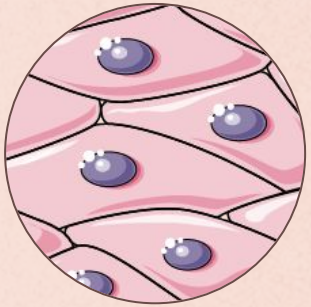
Data augmentation

In our testing between our typical model without any adjustments like data augmentation or transfer learning that has known to significantly increase our accuracy in these models. While our model accuracy usually averages around 41% the same training model but with data augmentation can be seen to improve greatly at the accuracy average of 53.1%

Transfer learning

As transfer learning allow our machine to learn from previous experience, it helps the training process to become much faster and resource efficient because they have already learned the previous features without relearning it again. Our accuracy has seen a significant improvement of 86.3% accuracy.

Conclusions



Summary

Our CNN model has been trained using labeled microscopic images of colorectal tissues and can automatically classify colorectal tissues with a 86% accuracy.

Real-world application

Our model can be used to make the process of classifying colorectal tissue types easier, reduce diagnostic variability, and help researchers improve results in their studies.

